

## Part 1: Q1 (concurrency)

Concurrency in Healthy Habits occurs when more than one request tries to manipulate the same information concurrently. An example of concurrency issues in Healthy Habits is when a user opens two browser tabs with the calendar task day page open, and the user modifies or completes the same calendar task on each tab simultaneously. Another possible concurrency issue is when a user double-clicks on the button of a web form and submits the update request twice before the other request gets executed. In such situations, the application can process the latest request received by the server, hence overwriting a newer entry with the older one.

Though Spring Boot and Tomcat support concurrent requests, it is imperative to mention that the processing of those requests does not resolve the concurrency issues related to the business requirements of Healthy Habits. For instance, when updating calendar tasks, there is always a need for validation, ownership check, and database transactions that can help eliminate repetition, overwrite, or interference from another user. This requirement arises because, unlike temporary data in other apps, calendar tasks reflect user plans and progress in Healthy Habits.

This indicates that request concurrency took place during the profiling process. When the 25-users load was applied on IntelliJ using JMeter, there were some `http-nio-8080-exec-*` request threads running concurrently. This proves that there was concurrency within the application. Nonetheless, this cannot conclusively prove that the only reason for low performance was concurrency. This simply demonstrates that performance can be slowed down by shared server resources, including sessions, controllers, databases, and Thymeleaf templates.

## Part 1: Q2 (indexing)

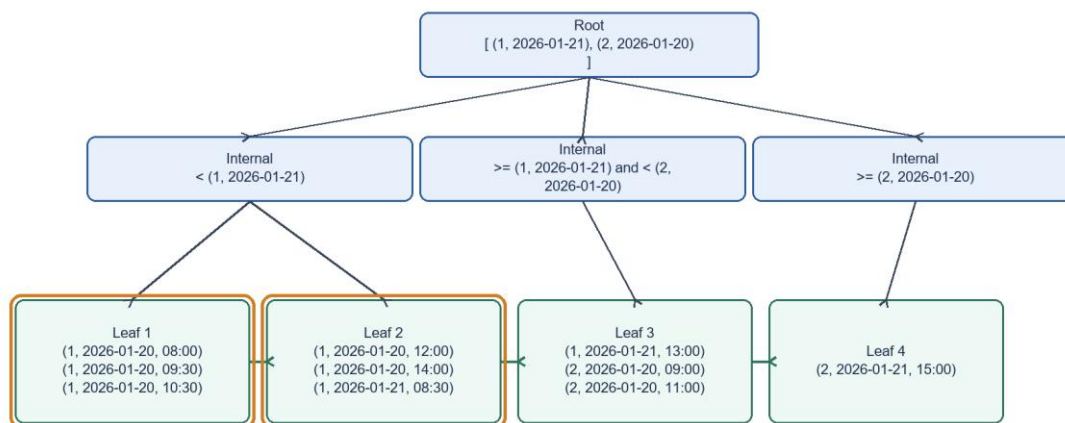
For a calendar day query, tasks have to be fetched for one user for one date, sorted by time. The created index is `calendar_tasks(user_id, date, time)`, which corresponds to how the calendar day view displays data: `user_id` is needed to specify the right user; `date` is needed to narrow down the results to one day; and `time` is required to show results in order. Compared to the previous index of `user_id`, here we use three columns in one index which can satisfy both `user_id` and `date` criteria used in the day view.

Here we present a B+ tree with  $m=4$  and at least 10 ordered composite values, as an illustrative diagram rather than actual internal storage for this particular index created for H2 database. If we want to fetch records where `user_id = 1` and `date = 2026-01-20`, then we should use the B+ tree from the root all the way to the leaves with the right values. Scanning all table rows would take  $O(n)$  because we don't know whether our record can be found in one pass through. In contrast, looking up in the tree is about  $O(\log n)$ .

The leaves within a B+ tree have an ordering mechanism, which allows each leaf node to link up to other nodes within the same level. This ordering property would be beneficial for the calendar page since once the first instance of a matching key is found, all the records from neighboring leaves would be accessed to retrieve all tasks matching the selected date.

### B+ tree for `calendar_tasks(user_id, date, time)`, $m = 4$

Illustrative index structure for the calendar day query, not the exact physical H2 plan



Search example: `user_id = 1` and `date = 2026-01-20`

The tree narrows to matching composite keys, then the linked leaves provide the day's times in order.

A full table scan checks each row  $O(n)$ ; a B+ tree lookup follows a short path through the tree, approximately  $O(\log n)$ .

Figure 1: Illustrative B+ tree for `calendar_tasks(user_id, date, time)`,  $m = 4$ .

## Part 2: monitoring and profiling with analysis

I tested the same route using JMeter with and without the addition of an index: `/calendar/day/2026-01-20`. The test was conducted under 1, 5, 10, and 25 users utilizing the demo account. The errors for the final comparison tests were zero

percent, so the timings were calculated based on the number of successfully performed requests, not unsuccessful connections. In general, the AFTER testing results were slightly worse than before, not better.

The difference for 1 user was 81.47 ms to 83.77 ms. For 5 users, it was 45.94 ms to 46.47 ms. For 10 users, it was 38.33 ms to 41.26 ms. Finally, the average value for 25 users became 37.81 ms to 39.69 ms; the 95th percentile was 92 ms to 94 ms. These changes are minor, which does not provide any evidence about the efficiency of the index added locally.

Profiler evidence also supports this result. As can be seen in the profiler screenshot taken in IntelliJ IDE, there were several http-nio-8080-exec-\* request threads working while running this app with 25 users, indicating that the app indeed processed multiple requests. Nonetheless, there were not enough threads with a database bottleneck, as the workload is distributed between different request processing threads. The page request involves Spring Security/session handling, controller processing, database work, and Thymeleaf processing, meaning that the newly created index affects only some of the processes.

This experiment is helpful in avoiding a distorted result, since it clearly demonstrates that, while the index structure is valid for this query, the local H2 testing failed to prove any performance gain.

| Users | Samples | Before avg | After avg | Change | Before p95 | After p95 | Verdict                 |
|-------|---------|------------|-----------|--------|------------|-----------|-------------------------|
| 1     | 30      | 81.47      | 83.77     | -2.82% | 327        | 357       | Slightly slower         |
| 5     | 150     | 45.94      | 46.47     | -1.15% | 106        | 108       | Similar/slightly slower |
| 10    | 300     | 38.33      | 41.26     | -7.64% | 95         | 99        | Slower                  |
| 25    | 750     | 37.81      | 39.69     | -4.97% | 92         | 94        | Slightly slower         |

Table 1: JMeter before/after results for the calendar day route.

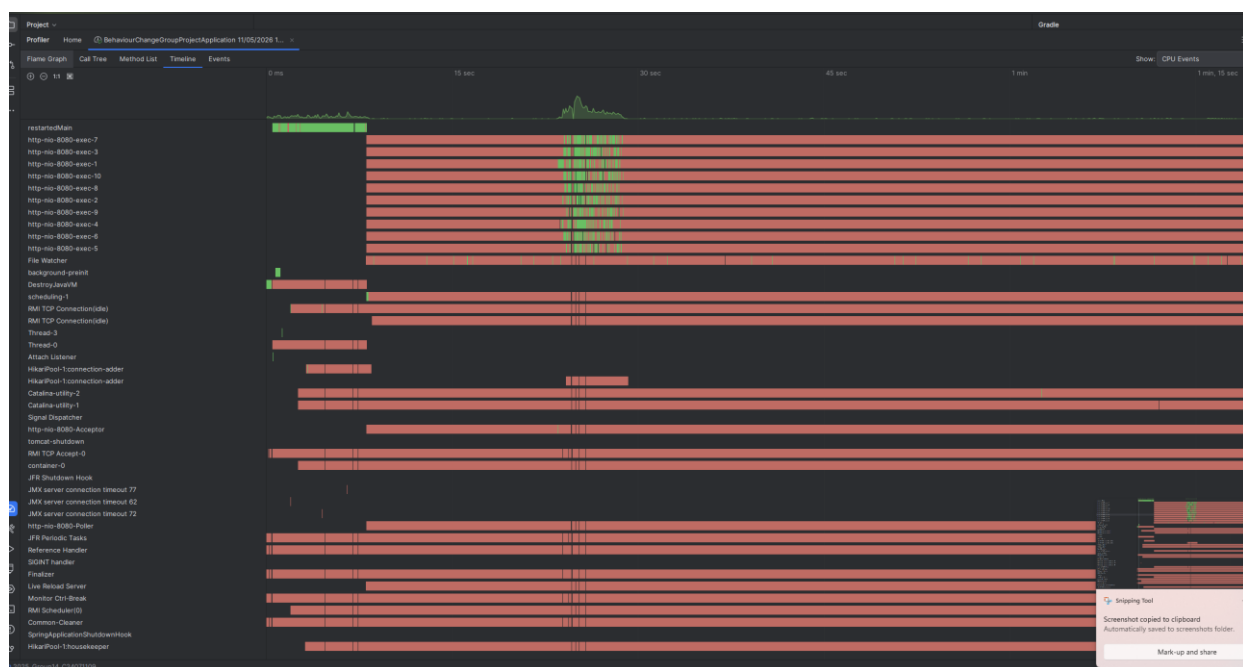


Figure 2: IntelliJ profiler evidence from the 25-user JMeter run.

## Part 2: explain plans analysis

Filtering the calendar\_tasks by the demo user and date 2026-01-20 and sorting it on the basis of time was done through the explain-plan query. In the absence of the composite index, H2 selected the user\_id foreign-key index path of calendar\_tasks table. In addition to that, the username subquery used the unique username index. This is an indication that the original query used some beneficial index access paths for getting user information rather than full table scans.

In presence of the composite index in schema, the index appeared in both schema files. In this context, it is necessary to mention that H2 selected the existing user\_id index path in the case of small seeded dataset. The significance of this observation lies in the fact that H2 never uses the idx\_calendar\_tasks\_user\_date\_time index explicitly. One possible reason can be that the local dataset was small and user\_id index was sufficiently selective.

It is consistent with what we saw in the results from the JMeter tool and the profiler. If the time taken for the request was not due to scanning lots of rows from the calendars table, then even having a more targeted index would not affect the timing of the page load. This explain plan confirms that an accurate analysis can be made, that the index matches the query pattern, but the performance increase was not verified with the local H2 database test.

## Part 2: recommendations and implementation

The recommended index creation was as follows:

```
CREATE INDEX IF NOT EXISTS idx_calendar_tasks_user_date_time
ON calendar_tasks (user_id, date, time);
```

This recommendation has been implemented in the files `src/main/resources/schema.sql` and `src/test/resources/schema.sql` to ensure consistency between the schema definitions of the main and test databases. The relevant feature branch is `feature/performance-cw2`. This recommendation can still be considered a suitable production-oriented suggestion since the PostgreSQL database with a greater amount of data would presumably have multiple tasks for multiple users and multiple dates. In such an environment, filtering tasks by both `user_id` and `date` is preferable to filtering just by `user_id`, while time helps implement the necessary ordering.

The primary problem is that the local effect of the H2 database was insignificant. The AFTER timing was close to normal but slower, and there was no indication that the H2 database chose the composite index in the explain plan. This information should be disclosed, not concealed.

The second suggestion is to make the calendar searches narrow. The day view must not unnecessarily query any dates or other tasks if it is restricted to a single day. In addition, future tests will need to be done on login/session cost separately from page-only cost, repeated tests following warm-up, and using a large PostgreSQL database-like sample set.

## References

`docs/cw2-evidence/jmeter/results-summary.md`

`docs/cw2-evidence/explain-plans/explain-comparison.md`

`docs/cw2-evidence/profiler-observations.md`

`docs/cw2-evidence/evidence-summary.md`